

✓ Curso: Modelos LLM (LOCAL vs API)

Objetivo

En este notebook vamos a entender los distintos tipos de modelos de lenguaje (LLMs), enfocándonos en dos formas principales de uso:

- Modelos vía API (OpenAI, Anthropic, etc.)
- Modelos locales (Llama, Mistral, etc.)

Dos formas de usar un LLM

Característica	API (Cloud)	Local
Infraestructura	Proveedor externo	Tu máquina
Costos	Pago por uso	Hardware + setup
Privacidad	Datos salen del entorno	Datos locales
Performance	Alta y optimizada	Depende del hardware
Modelos	GPT-4, Claude	LLaMA, Mistral

De que hablamos cuando decimos LLM local

Cuando ejecutás un LLM en tu máquina:

- El modelo está descargado como un archivo gigante
- Necesita CPU/GPU para ejecutar inferencias
- Procesa texto token por token
- Genera la siguiente palabra basada en probabilidades

En esencia:

 es un programa muy grande que corre en tu computadora

¿Qué partes tiene un LLM local?

Un modelo local típicamente incluye:

- **Modelo (red neuronal):** calcula probabilidades
- **Tokenizador:** convierte texto en números
- **Motor de inferencia:** ejecuta el modelo
- **Context window:** memoria limitada del texto

✓ 1. Modelo (red neuronal)

Es el componente principal.

- Es una red neuronal entrenada con grandes cantidades de texto
- Contiene los "patrones aprendidos" del lenguaje
- No almacena respuestas, sino relaciones estadísticas

 Es el núcleo de todo el sistema

✓ 2. Tokenizador

El modelo no entiende texto directamente.

Antes de procesar, el texto se convierte en tokens:

Ejemplo: "Hola mundo" → [15496, 995]

- Un token es un segmento de texto, puede ser un caracter solo, una fracción de una palabra, o una palabra completa.

El armado de un token en un tokenizador no es aleatorio ni estático; depende directamente de una combinación de reglas algorítmicas, el idioma del texto y el entrenamiento previo que haya tenido ese modelo.

Para entenderlo de forma sencilla, un tokenizador no "entiende" palabras, sino que fragmenta el texto en pedazos óptimos para que la inteligencia artificial los procese de la manera más eficiente posible.

Existen diferentes métodos para decidir dónde cortar un texto. Los más usados hoy en día en modelos de lenguaje (LLMs) son de subpalabras (subwords):

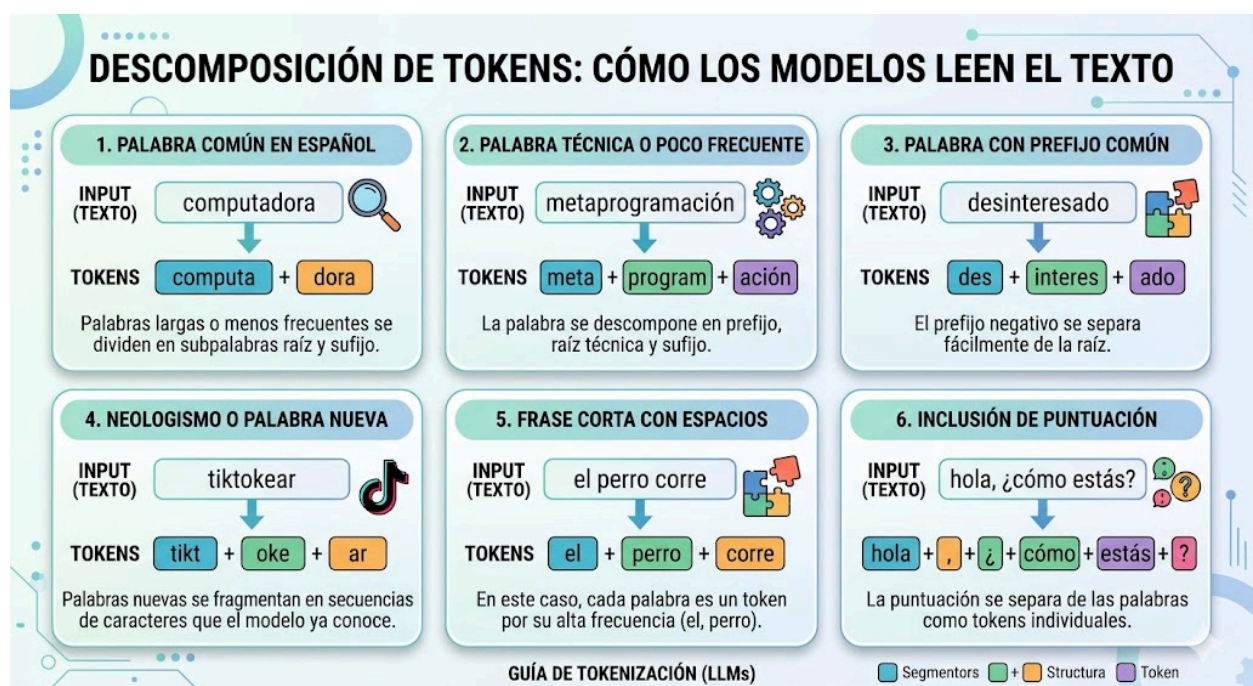
- BPE (Byte-Pair Encoding): Empieza dividiendo el texto en caracteres individuales y, de forma iterativa, fusiona los pares de caracteres o fragmentos que más se repiten juntos en el corpus de entrenamiento. Es el que usan modelos como GPT.
- WordPiece: Similar a BPE, pero en lugar de unir los pares que más se repiten, une los fragmentos que más aumentan la probabilidad estadística (verosimilitud) del modelo de lenguaje. Lo usa BERT.
- SentencePiece o Unigram: No necesita espacios en blanco obligatorios para separar palabras (ideal para idiomas como el japonés o chino). Trata todo el

texto como un flujo continuo de caracteres y va eliminando los tokens menos probables de un vocabulario gigante inicial.

Los tokenizadores se entrenan con bases de datos gigantescas. El armado del token dependerá de qué tan común sea una combinación de letras en ese grupo de datos:

Raíces y sufijos: En español o inglés, los prefijos (des-, sub-) o sufijos (-mente, -ción) suelen convertirse en tokens individuales porque se repiten en miles de palabras distintas.

Sesgo de idioma: Como la mayoría de los tokenizadores se entrenan con una cantidad masiva de texto en inglés, las palabras en inglés suelen armarse con menos tokens (a menudo una palabra entera es un token). En cambio, en español u otros idiomas, una misma palabra puede fragmentarse en 2, 3 o más tokens porque estadísticamente el algoritmo no la ve "tan seguido" como una unidad completa.



Una vez que ya tenemos un texto tokenizado (fraccionado en unidades minimas para su analisis.), procedemos a convertirlo en un lenguaje que pueda ser procesado por los modelos de LLM.

3. Embeddings

Cada token se convierte en un vector numérico.

Ejemplo conceptual: "perro" → [0.12, -0.88, 1.45, ...]

- Estos vectores representan significado
- Palabras similares tienen vectores cercanos

Un tokenizador solo sabe que la palabra "perro" es el número 4512 y "gato" es el 8923. No tiene idea de qué son. El embedding toma esos números y los ubica en un mapa multidimensional (un espacio vectorial) donde las cosas que se parecen o tienen relación se guardan cerca las unas de las otras.

Cada ID de token se busca en una matriz gigante y se transforma en un vector (una lista de números decimales, por ejemplo, de 768 o 1536 dimensiones). Cada uno de estos números decimales representa una "característica" abstracta del significado (si es un animal, si es un verbo, si tiene género masculino, si es doméstico, etc.). [gato] (ID 4512) $\rightarrow$ [0.23, -0.45, 0.89, 0.12, ...] [ratón] (ID 6314) $\rightarrow$ [0.21, -0.41, 0.85, -0.05, ...]

Cercanía semántica: Las palabras "gato" y "ratón" van a quedar flotando muy cerca una de la otra porque ambas son animales, mamíferos y suelen aparecer en el mismo contexto. En cambio, la palabra "computadora" quedará en una esquina completamente opuesta del mapa.



4. Inferencia

Es el proceso donde el modelo genera texto.

Funciona así:

- Recibe tokens de entrada
- Analiza contexto completo
- Calcula probabilidades del siguiente token
- Selecciona uno
- Repite el proceso

Cuando hablamos de Inferencia en un Modelo de Lenguaje (LLM), estamos hablando del momento en que la inteligencia artificial "piensa" y responde.

Es la etapa de ejecución en tiempo real, donde el modelo ya está entrenado (ya tiene sus "estudios" completos) y ahora se dedica exclusivamente a procesar lo que vos le escribís (el prompt) para predecir y generar la mejor respuesta posible, un token a la vez.

Desglosando esos puntos que mencionás para tu material, el proceso "bajo el capó" ocurre exactamente así:

1. Recibe tokens de entrada El modelo no lee palabras enteras. Como vimos antes, el prompt que vos escribís pasa por el tokenizador, se transforma en números (IDs de tokens) y luego en vectores (embeddings). Esos vectores entran en la red neuronal como el punto de partida.
2. Analiza el contexto completo Aquí entra en juego el motor principal de los modelos modernos: el mecanismo de Atención (Attention). El modelo no mira las palabras de forma aislada, sino que calcula la relación que tiene cada token con todos los demás de la frase.

Por ejemplo, si el texto dice "El banco estaba cerrado", el mecanismo de atención analiza si el contexto habla de dinero (un banco financiero) o de una plaza (un banco para sentarse).

3. Calcula probabilidades del siguiente token Esto es clave: un LLM no sabe lo que va a decir al final de la frase cuando empieza a escribir. Es un motor estadístico ultra avanzado.

Basándose en todo el contexto anterior, el modelo revisa todo su diccionario (su vocabulario de 50.000 o 100.000 tokens) y le asigna un porcentaje de probabilidad a cada uno de ellos para ver cuál es el que tiene más sentido matemático para continuar la frase.

4. Selecciona uno Una vez que tiene la lista de probabilidades, el modelo elige un token. No siempre elige el que tiene 100% de probabilidad (eso haría que las respuestas fueran idénticas, robóticas y aburridas).

Ahí es donde entran parámetros como la Temperatura:

Temperatura baja (ej. 0.2): El modelo es muy conservador y elige casi siempre el token más probable (ideal para programación o datos exactos).

Temperatura alta (ej. 0.8 o 1.0): El modelo arriesga más y elige tokens con menos probabilidad, lo que genera respuestas más creativas o variadas.

5. Repite el proceso (Ciclo Autorregresivo) Este paso es el que crea la magia de la escritura fluida. El token seleccionado se "pega" al final del texto anterior, y

todo el bloque completo vuelve a entrar al modelo para calcular el siguiente token.

Este bucle se repite una y otra vez hasta que ocurre una de dos cosas:

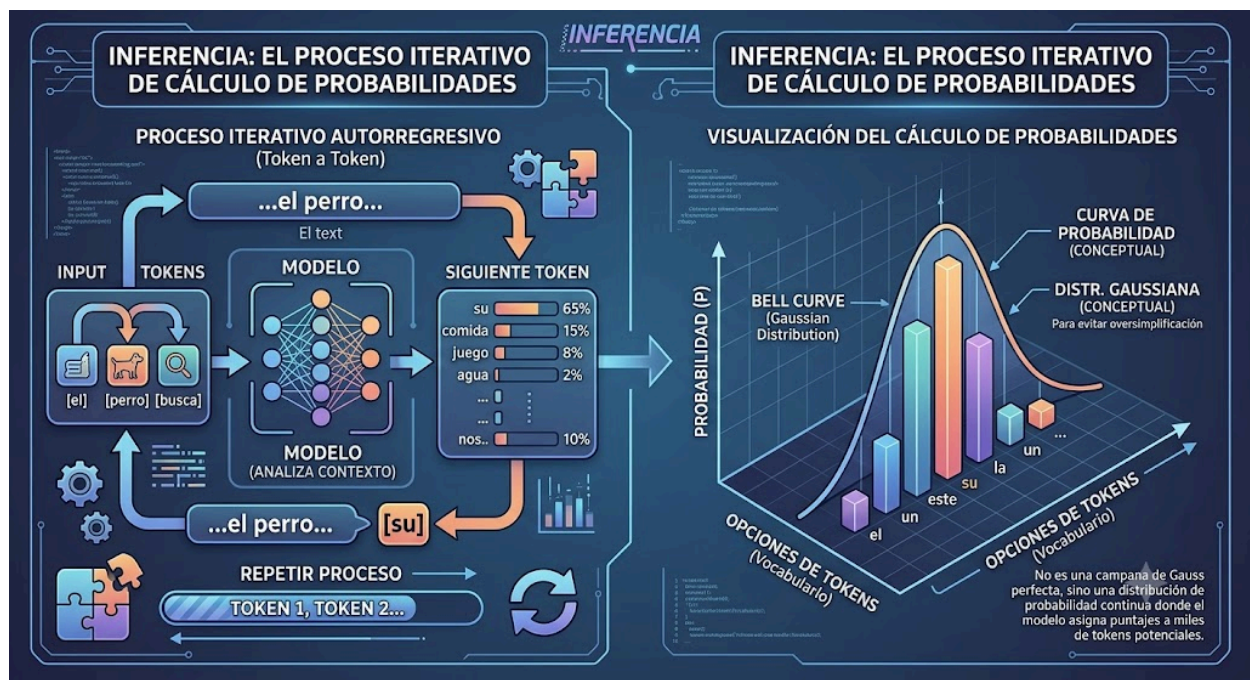
El modelo genera un token especial llamado EOS (End of Sentence / Fin de Secuencia), que le indica que la respuesta terminó.

Se alcanza el límite máximo de tokens configurado por el usuario.

Si el prompt de entrada es:

"El mejor amigo del hombre es el..."

- Ciclo 1: El modelo analiza el contexto → Calcula probabilidades (perro: 92%, gato: 5%, auto: 1%) → Selecciona: perro.
- Ciclo 2: La nueva entrada es: "El mejor amigo del hombre es el perro..." → Analiza contexto → Calcula probabilidades (. : 85%, ,: 10%, fiel: 3%) → Selecciona: . (punto final).
- Ciclo 3: Detecta el punto final o token de cierre → Detiene la inferencia y muestra el resultado. En resumen, la inferencia es un bucle de predicción estadística ultra veloz que ocurre a la velocidad en que ves aparecer las letras en la pantalla.



5. Ventana de contexto

El modelo no tiene memoria infinita.

Solo puede "ver" una cantidad limitada de tokens:

- Ej: 4K, 8K, 32K tokens
- Todo lo que queda fuera se pierde

Esto define cuánto puede “recordar” en una conversación

La ventana de contexto (o context window) es la cantidad máxima de texto que un modelo de inteligencia artificial puede “leer”, recordar y procesar al mismo tiempo en una sola interacción.

Basicamente seria la memoria a corto plazo del modelo. Todo lo que esté dentro de esa ventana se considera para el analisis mientras genera una respuesta; todo lo que quede afuera, simplemente deja de existir para la IA en ese momento.

Las ventanas de contexto se miden por unidades de tokens.

Si un modelo tiene una ventana de contexto de 8.000 tokens, significa que la suma de tu prompt (pregunta, archivos adjuntos, historial de la conversación) MÁS la respuesta que va a generar el modelo, no puede superar ese límite.

El tamaño de la ventana de contexto depende del costo computacional (La atención): Los modelos usan el mecanismo de Self-Attention (Autoatención).

Esto significa que cada token se compara matemáticamente con todos los demás tokens de la ventana. Si duplicás el contexto, el esfuerzo de procesamiento no se duplica, sino que crece de forma exponencial (N^2).

Procesar ventanas gigantes requiere un poder de cómputo y una memoria RAM de video (VRAM) brutales. Evolución tecnológica: Los primeros modelos (como el GPT-3 original) tenían ventanas muy chicas, de apenas 2.048 tokens (unas 3 páginas de texto). Hoy en día, técnicas avanzadas de compresión y optimización matemática permiten ventanas masivas: desde 128.000 tokens (unas 300 páginas) hasta 1 o 2 millones de tokens (capaces de procesar libros enteros, horas de audio o código fuente de aplicaciones completas en un solo segundo).

Existen actualmente distintas tecnicas que se aplican sobre los modelos mas grandes para simular una ventana de contexto enorme al guardar resúmenes del historial de conversación en bases vectoriales que son consultadas en tiempo de ejecución.



Como fin del proceso podemos decir que “Un LLM local es un sistema que convierte texto en tokens, los transforma en vectores, los procesa con una red neuronal y devuelve texto prediciendo token por token.”

Hasta este punto entendemos como se arman oraciones en un modelo LLM, pero podría surgir la duda

¿Cómo un LLM responde una pregunta?

El modelo no entiende preguntas entiende distribuciones de texto donde las preguntas suelen ser seguidas por respuestas

1. Recibe un prompt (texto)
2. Ese texto contiene contexto + estructura
3. El modelo detecta el patrón
4. Predice cómo debería continuar
5. Genera la respuesta token por token

¿Y por qué responde “bien”?

Porque fue entrenado con enormes cantidades de texto como:

Foros Libros Conversaciones Q&A Documentación

Entonces aprendió patrones como:

“¿Qué es X?” → definición

“Cómo hacer X” → pasos

“Comparar X vs Y” → tabla o análisis

✓ Modelos en local

En esta sección vamos a ver un paso a paso como montar un modelo local y realizar las primeras consultas simples, este procedimiento se puede realizar con distintos modelos ya sea para experimentar, para funciones distintas, o para realizar un benchmark de rendimiento. Hay que tener en cuenta que los modelos se deben de descargar de internet y requieren bastante espacio en disco y una vez ejecutado requieran una cantidad importante de memoria RAM.

La cantidad de RAM necesaria depende directamente del tamaño del modelo LLM que se pretenda ejecutar. El tamaño del modelo se suele medir en miles de millones de parámetros (por ejemplo, 7B para 7 mil millones de parámetros, 13B para 13 mil millones).

Una regla general muy aproximada para modelos no optimizados solía ser que se necesitaba un poco más de RAM que el tamaño del modelo en gigabytes (GB), multiplicado por el tamaño de cada parámetro (a menudo 2 bytes o más). Por ejemplo, un modelo de 7B parámetros donde cada parámetro ocupa 2 bytes necesitaría aproximadamente $7 \times 2 = 14$

$7 \times 2 = 14$ GB de RAM solo para los parámetros, más un extra para los cálculos y el sistema operativo.

Sin embargo, las técnicas modernas han mejorado significativamente la eficiencia

- Requisitos mínimos (modelos/experimentos pequeños): 8 GB de RAM podrían permitirle ejecutar modelos muy pequeños (por ejemplo, de 1B a 3B parámetros) o versiones altamente cuantificadas de modelos ligeramente más grandes, pero el rendimiento podría ser lento y tendrá que cerrar otras aplicaciones.
- Recomendado para modelos comunes (por ejemplo, 7B): 16 GB de RAM suele ser un punto de partida práctico para ejecutar cómodamente los modelos 7B cuantizados más populares junto con el sistema operativo y otras aplicaciones básicas. Ideal para modelos más grandes (p. ej., 13B+): 32 GB de RAM ofrecen mayor flexibilidad, lo que permite ejecutar modelos 13B bien cuantizados o incluso algunos más grandes (como modelos 30B+ con cuantización intensiva) con mayor fluidez. Además, proporciona más margen para el sistema operativo y otras tareas.
- Para entusiastas/modelos de mayor tamaño: 64 GB o más son beneficiosos si planea ejecutar modelos más grandes (30 mil millones o más) con menos

cuantización o experimentar con la ejecución de varios modelos o flujos de trabajo más complejos.

Para simplificar la instalación y pruebas de modelos nos valemos de herramientas desarrolladas para este fin, estos son gestores de modelos de lenguaje. Su función principal es permitirte descargar, configurar y ejecutar inteligencias artificiales de código abierto (como Llama, Gemma o Qwen) utilizando el hardware de tu propia computadora (procesador, placa de video y RAM), sin depender de internet ni de servicios pagos en la nube.

Entre los principales o mas comunes tenemos Ollama, LLMStudio, y text-generation-webui

Ollama está pensado principalmente para trabajar desde la consola de comandos (CLI) y como un servicio que corre en segundo plano. Es extremadamente popular entre desarrolladores por su minimalismo y velocidad.

Con una sola línea en la terminal (ejemplo: ollama run llama4)
descarga el modelo y te abre un chat ahí mismo.

Levanta de forma automática un servidor local que replica exactamente la estructura de la API de OpenAI. Esto te permite conectar tus modelos locales a herramientas de desarrollo (como extensiones de VS Code), frameworks de agentes de IA o interfaces web de terceros en pocos segundos.

Ideal para: Automatizaciones, integraciones de código, servidores headless (sin interfaz gráfica) y usuarios avanzados de terminal.

LM Studio (El centro de control visual)

A diferencia de Ollama, LM Studio ofrece una interfaz gráfica (GUI) impecable y muy pulida, diseñada para que cualquier persona (técnica o no) pueda gestionar IA local de forma visual.

Cuenta con un buscador integrado conectado directamente a Hugging Face (el gran repositorio de modelos abiertos).

Te dice qué versión de cada modelo es compatible con los GB de RAM o VRAM de tu sistema para evitar que tu máquina colapse.

Podés modificar la temperatura, ajustar el tamaño de la ventana de contexto moviendo un deslizador o configurar el desvío de capas de la red neuronal hacia la GPU de forma intuitiva. También incluye un servidor local para integraciones. Ideal para experimentar con configuraciones, probar modelos nuevos rápidamente, chatear de forma tradicional y para quienes prefieren no tocar la línea de comandos.

Característica	Ollama	LM Studio
Interfaz Principal	Consola de comandos (CLI)	Aplicación de escritorio (GUI)

Característica	Ollama	LM Studio
Curva de Aprendizaje	Muy baja (si sabés usar terminal)	Extremadamente baja (Familiar y visual)
Buscador de Modelos	Integrado en su propia librería online	Integrado directo a Hugging Face en la app
Consumo de recursos	Muy ligero (sin interfaz propia)	Moderado (la interfaz consume algo de men
Ideal para...	Desarrolladores y automatización backend	Usuarios finales y testing visual de modelos

Vamos a proceder con la instalación

Instalación de Ollama

Antes de ejecutar el código, instalar Ollama:

<https://ollama.com>

Las instalaciones van a diferir en el sistema operativo que se realice un ejemplo con windows es:

Descarga el instalador: Visita la página oficial de descargas de Ollama y haz clic en el botón Download for Windows para obtener el archivo OllamaSetup.exe.

Ejecuta el instalador: Haz doble clic en el archivo descargado y sigue el asistente (haz clic en Install).

Verifica que esté activo: Una vez instalado, verás el icono de una llama en la bandeja del sistema (junto al reloj en la barra de tareas).

Comprueba en tu terminal: Abre PowerShell o el Símbolo del sistema (CMD) y escribe ollama para confirmar que se instaló correctamente.

-Cómo descargar y ejecutar tu primer modelo Explora el catálogo de modelos de Ollama y elige uno (por ejemplo, llama3.2 o deepseek-r1).

En tu terminal (PowerShell o CMD), escribe el comando de ejecución, que a su vez descargará el modelo automáticamente:ollama run llama3.2

En Linux

Abre tu terminal y ejecuta el siguiente comando Download Ollama on Linux:curl -fsSL <https://ollama.com/install.sh> | sh

Luego descargar un modelo:

```
ollama pull llama3
```

```
---
```

▼ Iniciar un modelo

Vemos la linea de comandos disponibles

Available Commands: serve Start Ollama

create Create a model

show Show information for a model

run Run a model

stop Stop a running model

pull Pull a model from a registry

push Push a model to a registry

signin Sign in to ollama.com

signout Sign out from ollama.com

list List models

ps List running models

cp Copy a model

rm Remove a model

launch Launch the Ollama menu or an integration

help Help about any command

Vamos a ver modelos descargados en local

```
ollama list
```

NAME	ID	SIZE	MODIFIED
phi3:latest	4f2222927938	2.2 GB	4 weeks ago
llama3:latest	365c0bd3c000	4.7 GB	4 weeks ago
mistral:latest	6577803aa9a0	4.4 GB	4 weeks ago

En el siguiente ejemplo vamos a poner a correr desde nuestro terminal en local un modelo de LLM liviano como es phi3, nos dara un prom para poder interactuar con el modelo

```
$ ollama run phi3
```

-> Que es una LLM?

Una "LLM" podría referirse a varias cosas, pero comúnmente se usa para abreviar la frase en inglés "Low Level Language." Sin embargo, si estás hablando del contexto de inteligencia artificial y lenguaje natural (IA/NL), es probable que te refieran con

precisión a un modelo basado en una arquitectura de Transformer como GPT-3 desarrollado por Microsoft.

Un LLM (llamado así porque se entrena para comprender, generar texto y realizar tareas relacionadas principalmente mediante el uso de lenguaje), es un tipo poderoso modelo computacional que ha sido preentrenado en una gran cantidad de texto. Esta capacidad le permite ajustarlo (fine-tuning) para responder preguntas, generar contenido creativo o completar tareas específicamente relacionadas con el lenguaje natural.

Vamos a dar el siguiente paso, vamos a realizar la misma consulta pero mediante python.

Para esto creamos un archivo de prueba por ejemplo:

test.py

Con el siguiente código:

```
1 import requests
2
3 url = "http://localhost:11434/api/generate"
4
5 data = {
6     "model": "phi3",
7     "prompt": "Explica qué es un LLM en una frase",
8     "stream": False
9 }
10
11 # Usamos la librería request para poder comunicarnos mediante un p
12 response = requests.post(url, json=data)
13
14 # El output tiene un formato JSON
15 print(response.json()["response"])
```

Ejecutamos el archivo desde consola o desde el IDE de nuestra preferencia

->python test.py

Un modelo de lenguaje a gran escala (LLM) como GPT-3, diseñado para comprender y generar texto similar al humano.

¿Qué está pasando acá?

- `url`: Ollama expone una API local (como si fuera una API en internet)
- `model`: el modelo que descargamos
- `prompt`: el texto de entrada

- `stream=False`: queremos la respuesta completa de una vez

Relación con lo que vimos antes

Este código hace exactamente esto:

1. Envía texto (prompt)
2. Ollama lo tokeniza
3. El modelo genera tokens
4. Se reconstruye el texto
5. Se devuelve la respuesta

 Todo ocurre localmente

✓ De que otra forma podemos comunicarnos en local?

Existen librerías ya armadas para poder comunicarse de forma simple con con el modelo, como ejemplo:

- ollama

Podemos ahora crear un archivo test2.py con el código:

```
1
2 import ollama
3
4 response = ollama.chat(
5     model='phi3',
6     messages=[
7         {"role": "user", "content": "¿Qué es un LLM?"}
8     ]
9 )
10
11 print(response['message']['content'])
```

Recordar que si no se tiene la librería ollama instalada se deberá correr

`pip install ollama`

-Se recomienda que siempre que sea posible utilizar un entorno virtual por ejemplo venv para todas estas prácticas

->`python test2.py`

Un LLM (Large Language Model) es un tipo de inteligencia artificial especializada en el procesamiento y comprensión del lenguaje natural. Estos modelos están

entrenados con grandes cantidades de datos textuales, lo que les permite realizar una variedad de tareas relacionadas con la lingüística computacional, como traducción automática entre idiomas diferentes, generación de respuestas coherentes y contextualmente apropiadas a preguntas o temas específicos, completado de texto (como el que estás haciendo aquí), reconocimiento de voz y más. Los LLMs se han vuelto cada vez más prominently en aplicaciones como asistentes virtuales inteligentes (por ejemplo, ChatGPT) e interfaces interactivas para ayudar a mejorar la experiencia del usuario al procesamiento natural lenguaje humano-a-human.

¿Qué es `ollama.chat()`?

Es una función de la librería de Ollama que:

- Recibe una conversación estructurada
- La envía al modelo local
- Ejecuta todo el pipeline interno (tokenización + inferencia)
- Devuelve la respuesta

“Mandá este mensaje al modelo y devolveme lo que genere”

¿Que funcion cumple `messages`?

```
messages = [ {"role": "user", "content":  
              "¿Qué es un LLM?" } ]
```

`messages` es una lista de diccionarios que le vamos a pasar al modelo, ¿y que contiene?

✓ # role y content

Hay 3 roles típicos:

- "user" → lo que pregunta el usuario
- "assistant" → respuestas anteriores del modelo
- "system" → instrucciones globales

-
- `content` es el texto que ingresa al modelo por parte del "role" indicado
-

¿Porque hacemos `print(response['message']['content'])` ?

El objeto devuelto `response` es del tipo `<class 'ollama._types.ChatResponse'>`

Si imprimimos completamente el response tendremos, no solo la respuesta a nuestro prompt, sino tambien los parametros con lo que se ha devuelto:

role='assistant' content='Un LLM (Large Language Model) es una inteligencia artificial avanzada diseñada para entender, generar y procesar el lenguaje humano de manera similar a los humanos. Estos modelos utilizan grandes cantidades de datos textuales para aprender la estructura del idioma y pueden realizar tareas tales como traducción automática, conversación en inteligencia artificial (también conocida como chatbots), síntesis acústica-a-lingüística o escena a lenguaje natural y mucho más. Suelen ser capaces de mantener un contexto durante una interacción prolongada e incluso pueden generar texto que es coherente y relevante para el tema discutido. Los LLMs son componentes clave en muchas aplicaciones actuales como asistentes virtuales, chatbots educativos y herramientas de aprendizaje personalizado.' thinking=None images=None tool_name=None tool_calls=None

Tambien es factible comunicarse con el modelo con la funcion generate

```
1 response = ollama.generate(  
2     model='phi3',  
3     prompt="¿Qué es un LLM?"  
4 )  
5  
6 print(response['response'])
```

- `chat()` → estructura tipo conversación
- `generate()` → string directo

Ahora bien, tanto para chat como para generate podemos sumar otros parametros a la llamada al modelo:

```
1 response = ollama.generate(  
2     model='phi3',  
3     prompt="Explica qué es un LLM",  
4     options={  
5         "temperature": 0.7,  
6         "top_p": 0.9,  
7         "num_predict": 100  
8     }  
9 )
```

- temperature → creatividad vs precisión (0 a 1 donde 0 es mas creativo y factible de alucinar)
- top_p → diversidad de palabras

- num_predict → longitud máxima de respuesta

✓ CONSUMO DE API REMOTA

¿Qué cambia con una API?

En lugar de ejecutar el modelo localmente:

- 👉 enviamos el prompt a un servidor externo
- 👉 el modelo corre en la nube
- 👉 recibimos la respuesta

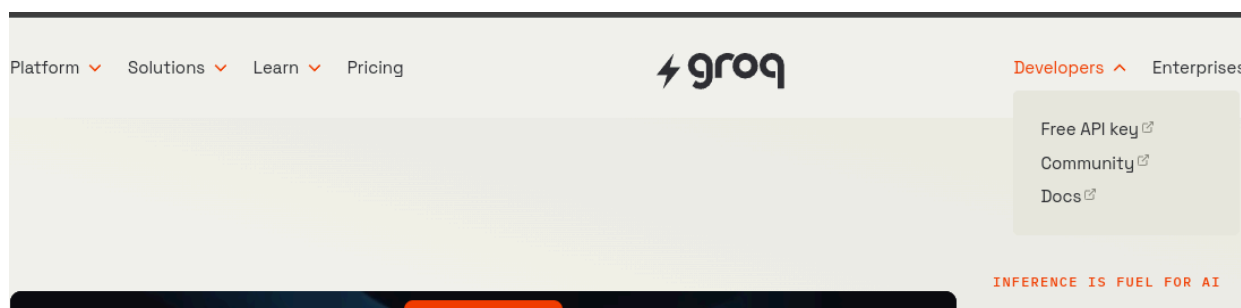
Ahora vamos a hacer lo mismo que hicimos con Ollama, pero usando un proveedor externo.

En este caso, usamos Groq <https://groq.com/>, una plataforma que permite ejecutar modelos de lenguaje en servidores optimizados.

El modelo ya no corre en nuestra máquina

Solo enviamos el prompt y recibimos la respuesta

Para poder usar Groq (disponible para pruebas de forma gratuita a la fecha Junio/26) deberemos crear una API KEY, una API KEY es una clave que usan las aplicaciones para poder comunicarse con recursos remotos como API's. Podemos pensar que es nuestro usuario y contraseña, pero es el de la aplicación o el código que estamos ejecutando.



✓ Instalamos la librería

```
!pip install groq
```

```
1 # Importamos la libreria para comunicarnos con la api
2 from groq import Groq
3
4 #Creamos la conexion con la funcion Groq
5 client = Groq(api_key="TU_API_KEY")
6
7 # client es nuestro objeto conexion para comunicarnos con la API
8 # utilizamos el metodo chat.completions.create para la consulta a
9 response = client.chat.completions.create(
10     model="llama3-8b-8192",
11     messages=[
12         {"role": "user", "content": "¿Qué es un LLM?"}
13     ]
14 )
15
16 print(response.choices[0].message.content)
```

Es importante resaltar que así como el usuario y contraseña no deben quedar expuestos en el código, tampoco la API KEY, para evitar esto hay técnicas recomendadas como el uso de variables de entorno, `.env`, o `key_vault` en el caso de google colab

Volviendo a lo que estamos viendo

Ambos códigos hacen lo mismo:

1. Envían un mensaje
2. El modelo genera texto
3. Devuelven una respuesta

La única diferencia es dónde corre el modelo

Esta diferencia no es menor, hay que tener en cuenta que Groq nos está facilitando su infraestructura para poder ejecutar la consulta, lo que implica que la información que le enviamos pasa por sus servidores y durante ese tiempo perdemos control y gobernanza de estos datos.

Esta situación se repite con cualquier proveedor de motores de LLM y en general con cualquier proveedor de servicios en la nube.

Por otra parte las pruebas con Groq al momento de redactar este instructivo son gratuitas, tienen límites de consumo, pero para hacer prácticas simples son suficientes.

Otras plataformas que se pueden usar son OpenAI, Gemini, Claude, todos estos modelos actualmente presentan como requerimiento la compra de tokens o de algún tipo de suscripción incluso para la realización de pruebas mínimas. Por esto recomiendo aprender sobre un entorno local o Groq para poder escalar luego a

plataformas pagas, donde el mecanimo no va a tener variaciones , mas alla que el nombre o el formato de algun metodo en particular.

Si te quedan preguntas no dudes en contactarme

arielmeragelman@gmail.com

Lic. Sebastian A. Meragelman